MEDI-SLOT

Architecture Diagram

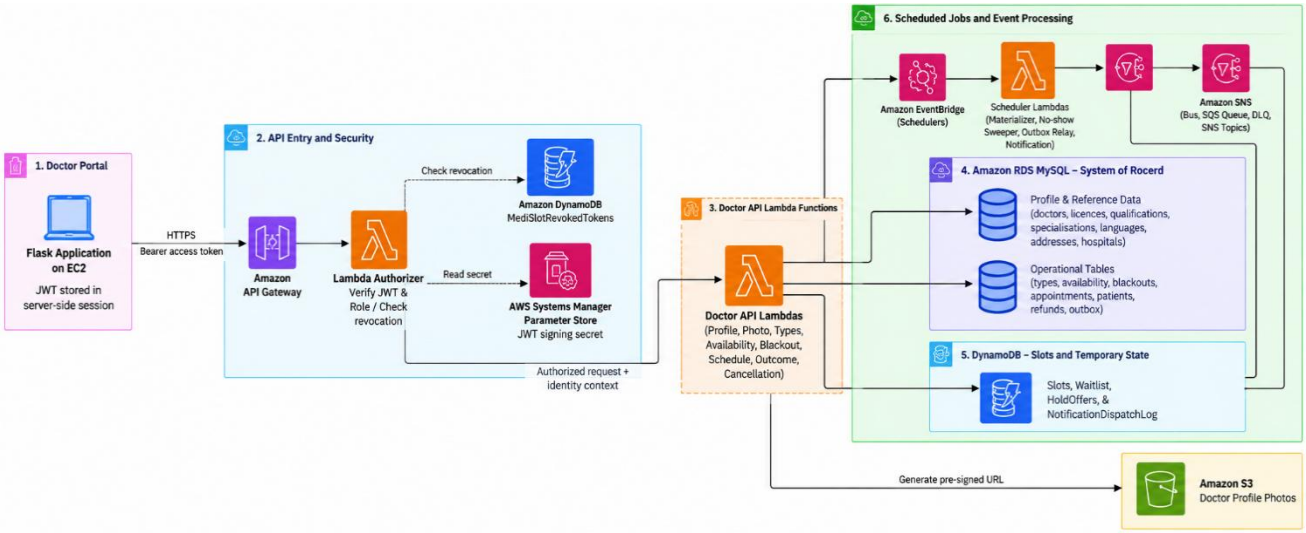


Figure: MediSlot Doctor Portal – System Architecture

Key Components

- **Doctor Portal:** Flask application hosted on Amazon EC2.
- **API Security:** API Gateway and Lambda Authorizer validate JWTs, roles, and token revocation.
- **Secret Management:** AWS Systems Manager Parameter Store stores the JWT signing secret.
- **Lambda Functions:** Handle doctor profiles, availability, scheduling, appointments, and cancellations.
- **Database:** Amazon RDS for MySQL stores doctor profiles and operational records.
- **Temporary State:** Amazon DynamoDB manages slots, waitlists, and temporary offers.
- **Event Processing:** Amazon EventBridge Scheduler triggers background jobs and workflows.
- **Notifications:** Amazon SNS, SQS, and DLQs support asynchronous notifications and failure handling.
- **Photo Storage:** Amazon S3 stores doctor profile photos using pre-signed URLs.

Confidential - Not for Public Consumption or Distribution

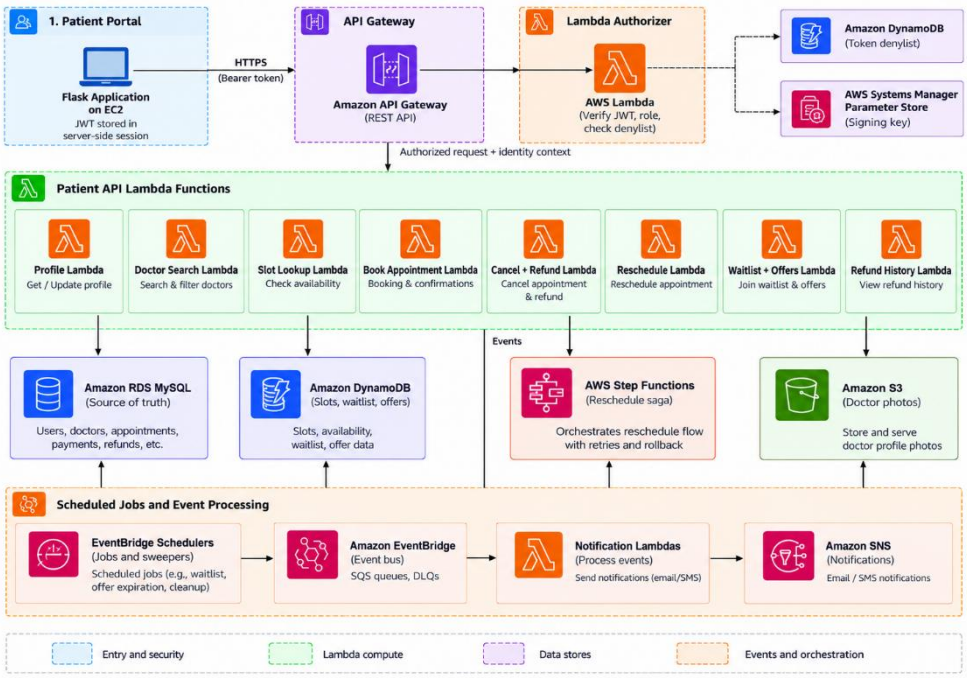


Figure: MediSlot Patient Portal – AWS System Architecture

Key Components

- **Patient Portal:** Flask application hosted on EC2 manages patient requests and JWT sessions.
- **API Security:** API Gateway and Lambda Authorizer validate JWTs, roles, and token revocation.
- **Patient API Lambdas:** Handle profiles, doctor search, slot lookup, appointments, cancellations, rescheduling, waitlists, and refunds.
- **Database Management:** RDS MySQL stores persistent application data, while DynamoDB manages slots, waitlists, and temporary offers.
- **Workflow Orchestration:** AWS Step Functions coordinates appointment rescheduling workflows with retries and rollback.
- **Photo Storage:** Amazon S3 stores doctor profile photos and supports secure access through pre-signed URLs.
- **Scheduled Processing:** EventBridge Scheduler triggers background jobs and cleanup tasks.
- **Event Notifications:** EventBridge, SQS, DLQs, notification Lambdas, and SNS support asynchronous event processing and email/SMS notifications

Confidential - Not for Public Consumption or Distribution

Services Used

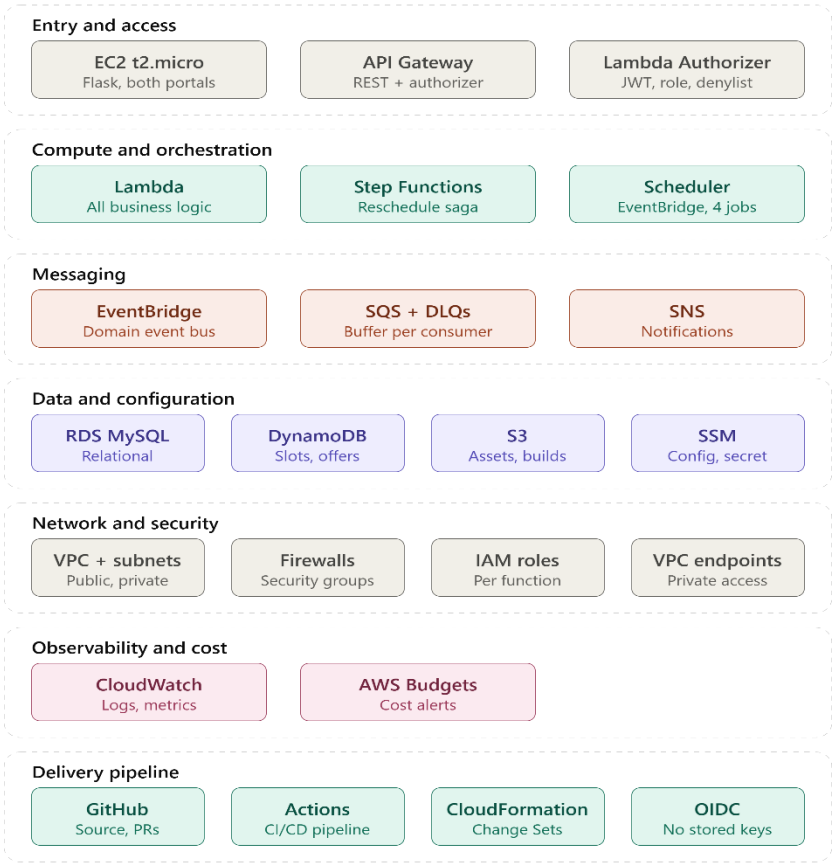


Figure: MediSlot – AWS Infrastructure and Services Architecture

- **Entry and Access:** EC2 hosts both portals; API Gateway and Lambda Authorizer secure API requests.
- **Compute and Orchestration:** Lambda handles business logic, Step Functions manages rescheduling, and EventBridge Scheduler triggers jobs.
- **Messaging:** EventBridge, SQS, DLQs, and SNS support event processing and notifications.
- **Data and Configuration:** RDS MySQL stores relational data, DynamoDB manages temporary state, S3 stores assets, and SSM manages secrets.
- **Network and Security:** VPC, subnets, security groups, IAM roles, and VPC endpoints provide network isolation and access control.
- **Monitoring and Cost:** CloudWatch collects logs and metrics, while AWS Budgets provides cost alerts.
- **CI/CD Pipeline:** GitHub Actions automates deployments using CloudFormation and OIDC authentication.

Confidential - Not for Public Consumption or Distribution

VPC LAYOUT

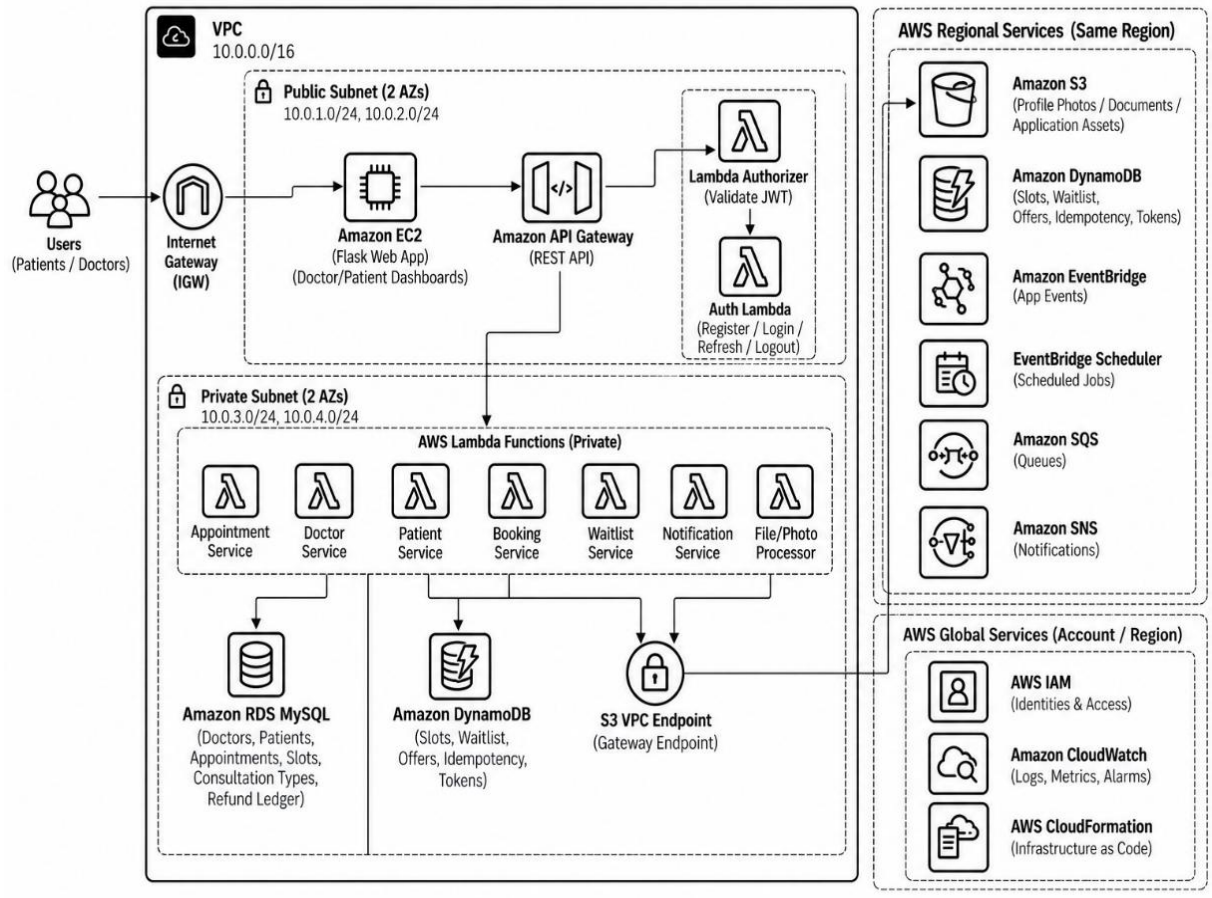


Figure: MediSlot – AWS VPC Network Architecture

Network Boundary & Ingress

- Users (Patients / Doctors): End-users accessing the web application and backend REST APIs.
- Internet Gateway (IGW): Allows public internet traffic to enter the Virtual Private Cloud (VPC).
- VPC (10.0.0.0/16): The primary isolated cloud network spanning two Availability Zones (AZs).

Confidential - Not for Public Consumption or Distribution

Public Subnet Components (10.0.1.0/24, 10.0.2.0/24)

- Amazon EC2 (Flask Web App): Hosts the web-based dashboards for patients and doctors.
- Amazon API Gateway (REST API): Entry point for backend API calls, managing endpoint routing and traffic.
- Lambda Authorizer: Validates JWT tokens for security before allowing API request execution.
- Auth Lambda: Handles authentication requests including registration, login, token refresh, and logout.

Private Subnet Components (10.0.3.0/24, 10.0.4.0/24)

- AWS Lambda Functions (Private): Isolated serverless microservices executing business logic for:
  - Appointment Service: Manages appointment scheduling logic.
  - Doctor Service: Handles doctor profiles and configuration.
  - Patient Service: Manages patient profile information.
  - Booking Service: Coordinates reservation and booking workflows.
  - Waitlist Service: Processes waitlisted requests and slot availability.
  - Notification Service: Triggers user communications.
  - File/Photo Processor: Manages asset uploads and image handling.
- Amazon RDS MySQL: Relational database storing structured relational data like doctors, patients, appointments, slots, consultation types, and refund ledgers.
- Amazon DynamoDB: NoSQL database inside the private subnet for high-speed retrieval of slots, waitlist offers, idempotency keys, and active session tokens.
- S3 VPC Endpoint (Gateway Endpoint): Allows private subnets to communicate directly with Amazon S3 without routing traffic over the public internet.

AWS Regional Services (Same Region)

- Amazon S3: Object storage for profile photos, user documents, and application assets.
- Amazon DynamoDB: Regional NoSQL table service for slots, waitlists, offers, idempotency keys, and session management.
- Amazon EventBridge: Event bus for real-time application event routing.
- EventBridge Scheduler: Runs time-based scheduled jobs and background tasks.
- Amazon SQS: Message queuing service for asynchronous task decoupling.
- Amazon SNS: Message publishing service for sending push notifications and alerts.

Confidential - Not for Public Consumption or Distribution

AWS Global Services (Account / Region)

- AWS IAM: Identity and Access Management for defining roles and execution policies.
- Amazon CloudWatch: Monitors system logs, operational metrics, and performance alarms.
- AWS CloudFormation: Infrastructure as Code (IaC) service used to provision and manage all infrastructure components automatically.

Event Topology

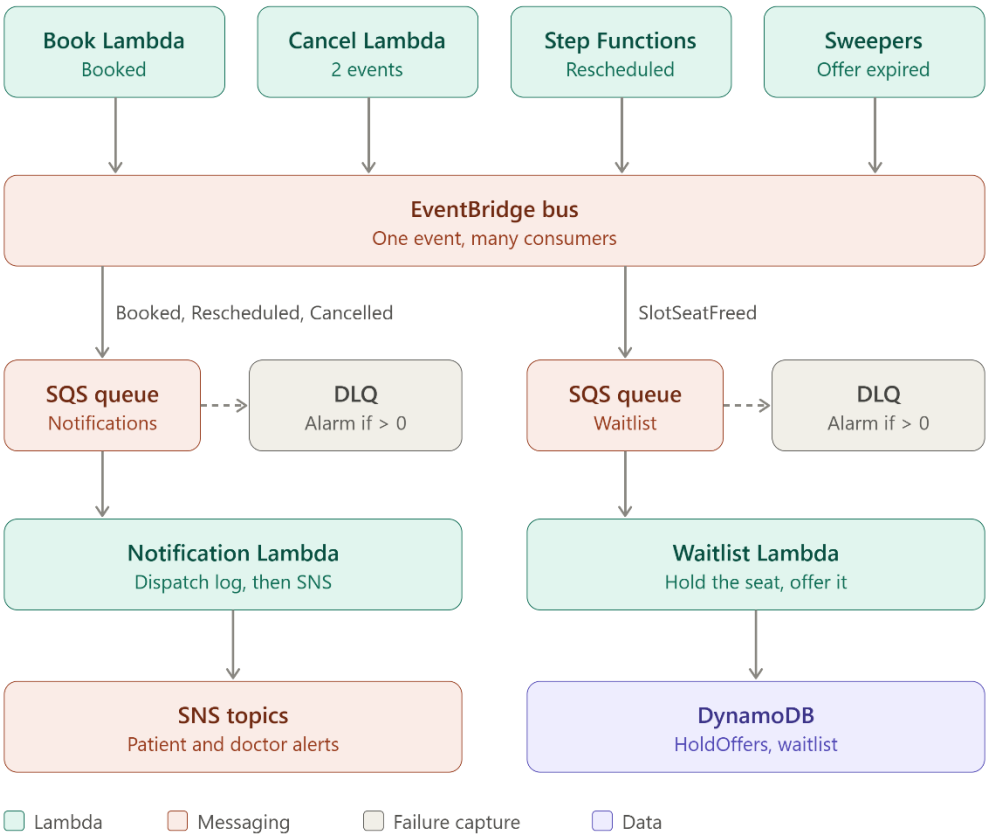


Figure: MediSlot – Event-Driven Architecture and Notification Workflow

Confidential - Not for Public Consumption or Distribution

- **Event Producers:** Book Lambda, Cancel Lambda, Step Functions, and sweepers publish events when appointments are booked, cancelled, rescheduled, or offers expire.
- **EventBridge Bus:** Receives events and routes them to multiple consumers based on event types.
- **Notification Queue:** SQS receives booking, rescheduling, and cancellation events for asynchronous notification processing.
- **Waitlist Queue:** A separate SQS queue receives SlotSeatFreed events to trigger waitlist processing.
- **Dead-Letter Queues (DLQs):** Capture messages that fail repeated processing. CloudWatch alarms can alert when a DLQ contains messages.
- **Notification Lambda:** Processes notification messages, records dispatch details in the dispatch log, and publishes notifications through SNS.
- **SNS Topics:** Deliver appointment-related alerts to patients and doctors.
- **Waitlist Lambda:** Processes freed slots, holds available seats, and creates offers for eligible waitlisted patients.
- **DynamoDB:** Stores HoldOffers and waitlist data to manage temporary seat reservations and offer status.

Event Catalogue

Common envelope (every event)

```
{
  "source": "medislot.booking",
  "detail-type": "AppointmentBooked",
  "detail": {
    "eventId": "uuid",
    "eventVersion": 1,
    "occurredAt": "2026-11-10T09:30:00Z",
    "correlationId": "uuid",
    "data": { }
  }
}
```

Confidential - Not for Public Consumption or Distribution

Field	Why it exists
eventId	Unique per event. Consumers use it for deduplication because SQS delivers messages at least once.
eventVersion	Allows payload changes without breaking existing consumers; supports API versioning.
occurredAt	UTC timestamp indicating when the event occurred.
correlationId	Created at the entry point and propagated through logs and events to trace a booking end to end
data	Contains the event-specific fields relevant to the particular event.

The Events

Event	Source	Published by	When
AppointmentBooked	medislot.booking	Book Lambda, Accept-offer Lambda	After the RDS appointment row is committed
AppointmentRescheduled	medislot.reschedule	Step Functions (last step)	Only on a fully successful saga
AppointmentCancelled	medislot.cancellation	Cancel Lambda ,doctor slot-cancel Lambda	After status change and refund ledger row are committed
SlotSeatFreed	medislot.slots	Cancel Lambda; reschedule saga (release step)	After the seat is released in DynamoDB
WaitlistOfferIssued	medislot.waitlist	Waitlist Lambda; expiry sweeper	After the hold offer is created
WaitlistOfferExpired	medislot.waitlist	Expiry sweeper	After the offer is marked EXPIRED

Confidential - Not for Public Consumption or Distribution

RDS SCHEMA

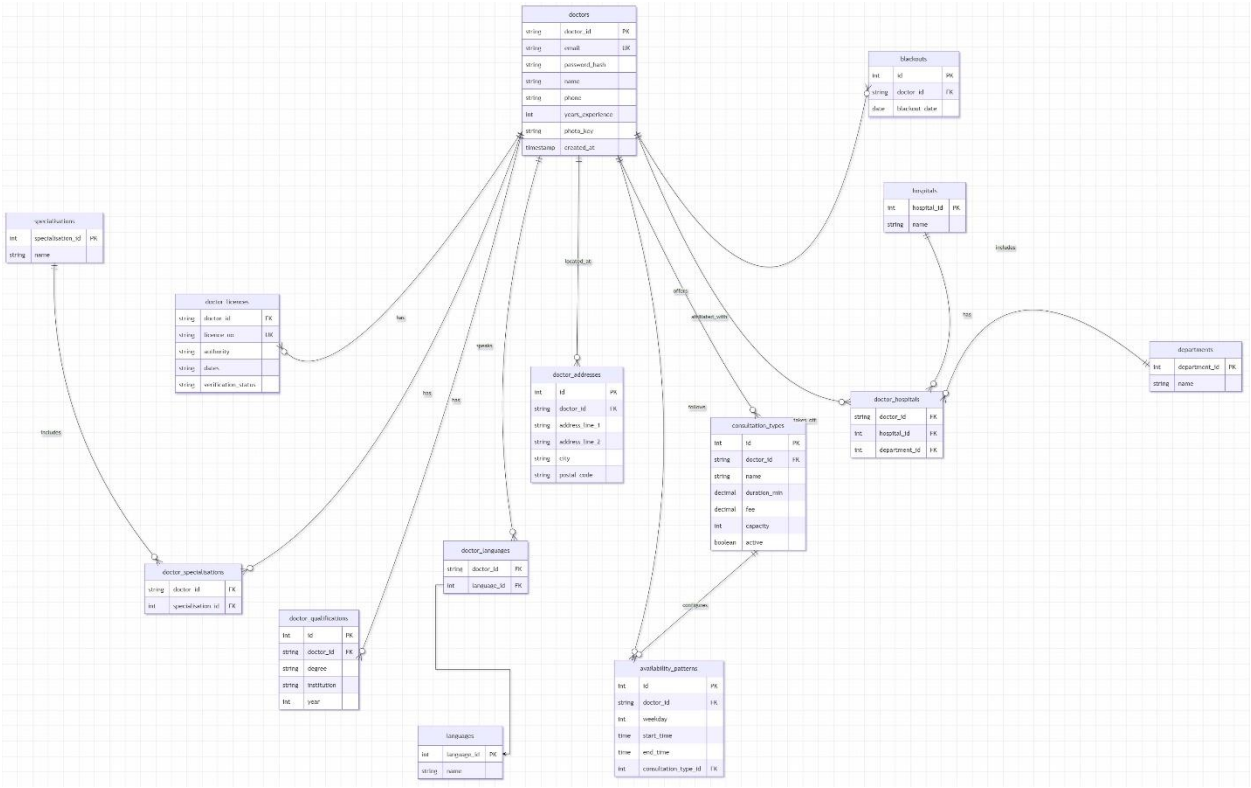


Figure: MediSlot – Doctor Database Entity-Relationship Diagram

Table-Wise Schema

Table	Columns and types	Keys / Indexes
doctors	doctor_id INT, user_id INT, email STRING, password_hash STRING, name STRING, phone STRING, years_experience INT, photo_url STRING, consultation_fee DECIMAL, is_active BOOLEAN, created_at TIMESTAMP	PK: doctor_id; UK: user_id

hospitals	hospital_id INT, name STRING, address STRING	PK: hospital_id
departments	department_id INT, name STRING	PK: department_id
doctor_hospitals	doctor_id INT, hospital_id INT, department_id INT	Composite PK: doctor_id, hospital_id; FKs to doctors, hospitals, departments
blackouts	id INT, doctor_id STRING, date DATE, blackout_date DATE	PK: id; FK: doctor_id
languages	language_id INT, name STRING	PK: language_id
doctor_languages	doctor_id INT, language_id INT	FKs to doctors and languages; composite PK recommended
doctor_qualifications	id INT, doctor_id INT, degree STRING, institution STRING, year INT	PK: id; FK: doctor_id
specializations	specialization_id INT, name STRING	PK: specialization_id
doctor_specializations	doctor_id INT, specialization_id INT	Composite PK; FKs to doctors and specializations
doctor_licences	doctor_id INT, licence_no STRING, authority STRING, states STRING, verification_status STRING	PK/FK: doctor_id; UK: licence_no
doctor_addresses	id INT, doctor_id INT, address_line_1 STRING, address_line_2 STRING, city STRING, postal_code STRING	PK: id; FK: doctor_id
consultation_types	id INT, doctor_id INT, name STRING, duration_min INT, fee_amount DECIMAL, capacity	PK: id; FK: doctor_id

Confidential - Not for Public Consumption or Distribution

	INT, is_active BOOLEAN	
availability_patterns	id INT, doctor_id INT, weekday INT, start_time TIME, end_time TIME, consultation_type_id INT	PK: id; FKs to doctors and consultation types

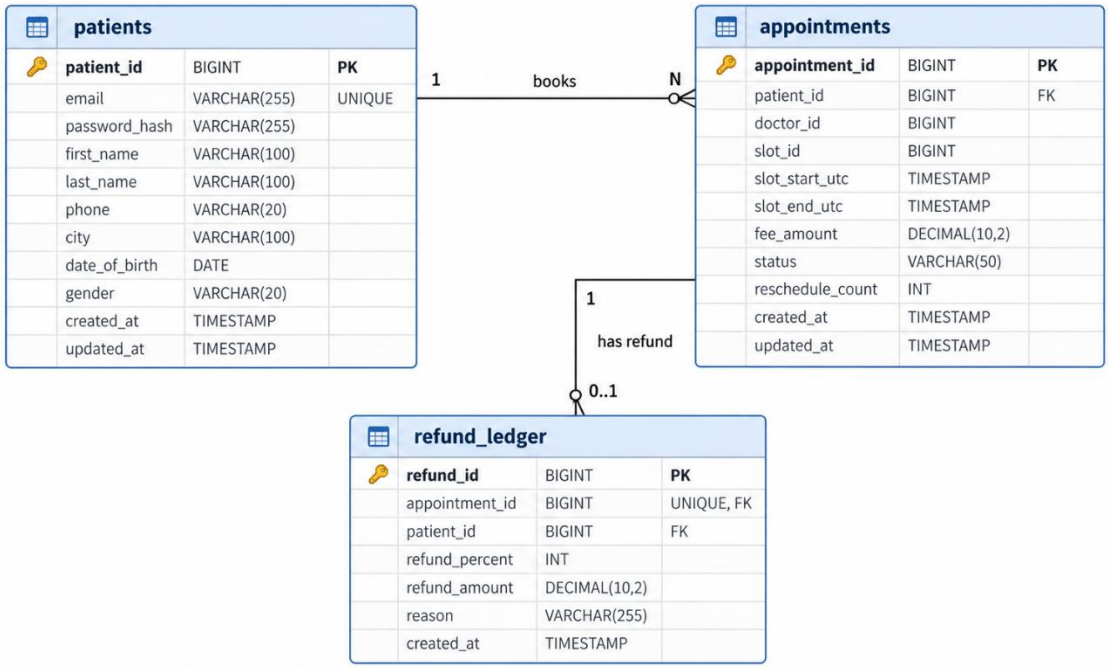


Figure: MediSlot – Patient, Appointment, and Refund Database ER Diagram

- **Patients:** Stores patient details, login credentials, contact information, and timestamps. patient_id is the primary key, and email is unique.
- **Appointments:** Stores appointment details, patient and doctor IDs, slot timings, fees, status, rescheduling count, and timestamps. appointment_id is the primary key, and patient_id references patients.
- **Refund Ledger:** Records refunds associated with appointments, including refund percentage, amount, reason, and creation time. refund_id is the primary key; appointment_id is a unique foreign key, allowing at most one refund record per appointment.
- **Relationships:** One patient can book multiple appointments. Each appointment can have zero or one refund record

Confidential - Not for Public Consumption or Distribution

DynamoDB Schema

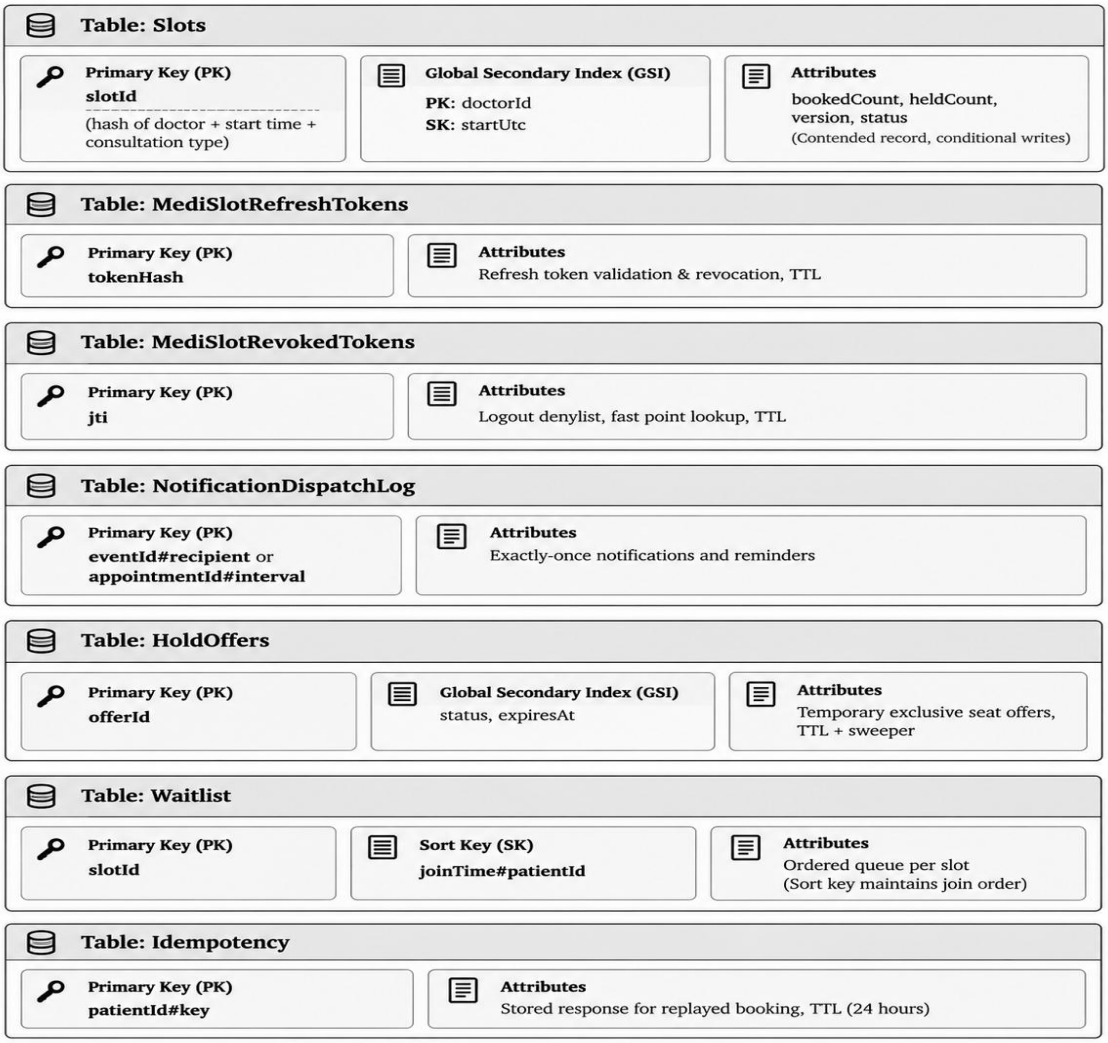


Figure: MediSlot – DynamoDB Table Schema and Key Design

Table descriptions

- **Slots:** Stores slot availability, booking counts, hold counts, status, and version. The GSI enables querying slots by doctor and start time.
- **MediSlotRefreshTokens:** Stores refresh-token validation and revocation information with a TTL.
- **MediSlotRevokedTokens:** Maintains a token denylist for logout and revocation checks.
- **NotificationDispatchLog:** Tracks notification dispatches to prevent duplicate notifications and reminders.
- **HoldOffers:** Stores temporary seat offers, with a GSI for querying by status and expiry time. TTL and sweepers handle expiration.
- **Waitlist:** Maintains the waiting order of patients for each slot.
- **Idempotency:** Stores booking responses to prevent duplicate processing when a request is retried, with a 24-hour TTL.

Confidential - Not for Public Consumption or Distribution

What are the different keys?

Key	Meaning
Partition Key (PK)	Identifies the item and determines its partition in DynamoDB.
Sort Key (SK)	Orders items within the same partition and can support range queries.
Primary Key	Uniquely identifies an item; can be a partition key alone or a partition key combined with a sort key.
Global Secondary Index (GSI)	Provides an alternative key structure for querying data using different attributes.
TTL (Time to Live)	An expiry attribute that allows DynamoDB to automatically delete expired items asynchronously.
Unique/composite key pattern	A key such as patientID key combines values to create a distinct identifier for a particular operation.

Concurrency Strategy

The Contended Resource

Attribute	Meaning
slot_id (PK)	doctorId#startUtc
capacity	1 for Quick, Standard and Detailed consults; 2 to 10 for Health Camp Sessions
taken_count	Seats unavailable to the public: booked seats plus seats under a live hold offer
booked_count	Confirmed appointments (reporting only)
version	Starts at 0 and increases by 1 on every successful write
slot_start_utc, status	Used to reject past or cancelled slots inside the same write

The invariant is 0 <= taken_count <= capacity. All writers must preserve it, and a conditional write is the only mechanism allowed to change taken_count.

Writers to this item: booking, patient cancellation, reschedule claim and release, hold-offer issue and expiry, offer acceptance, and doctor slot cancellation

Confidential - Not for Public Consumption or Distribution

The conditional write

UpdateExpression:

```
SET taken_count  = taken_count  + :one,
    booked_count = booked_count + :one,
    version      = version  + :one
```

ConditionExpression:

```
attribute_exists(slot_id)
AND version = :expectedVersion
AND taken_count < capacity
AND slot_start_utc > :nowUtc
AND slot_status = :open
```

ReturnValuesOnConditionCheckFailure: ALL_OLD

Condition	**Protects against**
taken_count < capacity	Overbooking. This is the hard guarantee, evaluated atomically by DynamoDB
version = :expectedVersion	Writing from a stale read. It makes every concurrent change visible as a conflict, and it is what the brief requires
slot_start_utc > :nowUtc, slot_status = :open	Booking a past or cancelled slot between the read and the write

Other writers use the same pattern with different conditions:

Operation	**Update**	**Condition (besides the version check)**
Release (cancel, reschedule release)	taken_count - 1, booked_count - 1	taken_count > 0
Issue hold offer	taken_count + 1	taken_count < capacity
Accept offer	booked_count + 1 (taken_count unchanged, since the hold already counted)	Offer is ACTIVE and unexpired (checked in the offer record, same transaction)
Expire offer, nobody waiting	taken_count - 1	taken_count > 0

Confidential - Not for Public Consumption or Distribution

Conflict Handling And Retry

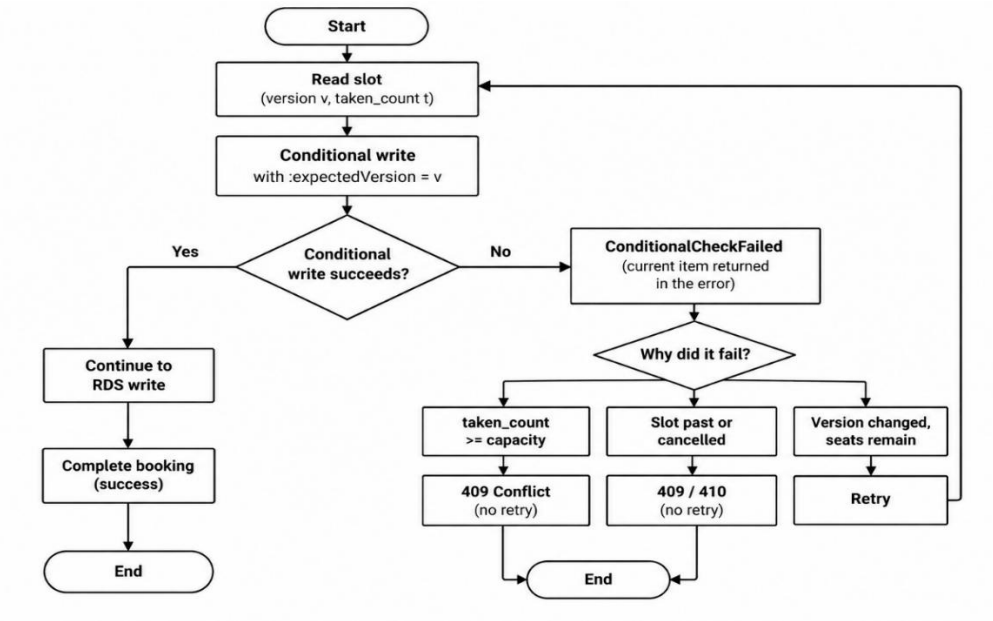


Figure: MediSlot – Slot Booking Concurrency Control and Retry Workflow

Initial State & Attempt

- **Start:** Initiates the slot reservation/booking process.
- **Read slot (version v, taken_count t):** Retrieves the current state of the requested slot, capturing its version number (v) and current booked count (t).
- **Conditional write with :** expectedVersion = v**:** Attempts to update the slot record on the condition that the version remains unchanged (v).

Decision Path

- **Conditional write succeeds?** Evaluates whether the optimistic locking check passed or failed.

Success Path (Yes)

- **Continue to RDS write:** Proceeds to write the confirmed transaction data to the relational database (RDS).
- **Complete booking (success):** Finalizes the booking process successfully.
- **End:** Terminates the execution workflow.

Confidential - Not for Public Consumption or Distribution

Failure Handling (No)

- **ConditionalCheckFailed:** Triggers an error returning the updated state of the item.
- **Why did it fail?** Evaluates the cause of the condition failure into three scenarios:

  - taken_count >= capacity**:** Returns a **409 Conflict** (no retry) due to fully booked capacity.
  - **Slot past or cancelled:** Returns a **409 / 410** status (no retry) because the slot is no longer active or valid.
  - **Version changed, seats remain:** Loops back to **Retry** by re-reading the slot and attempting the write again.

Reschedule Saga

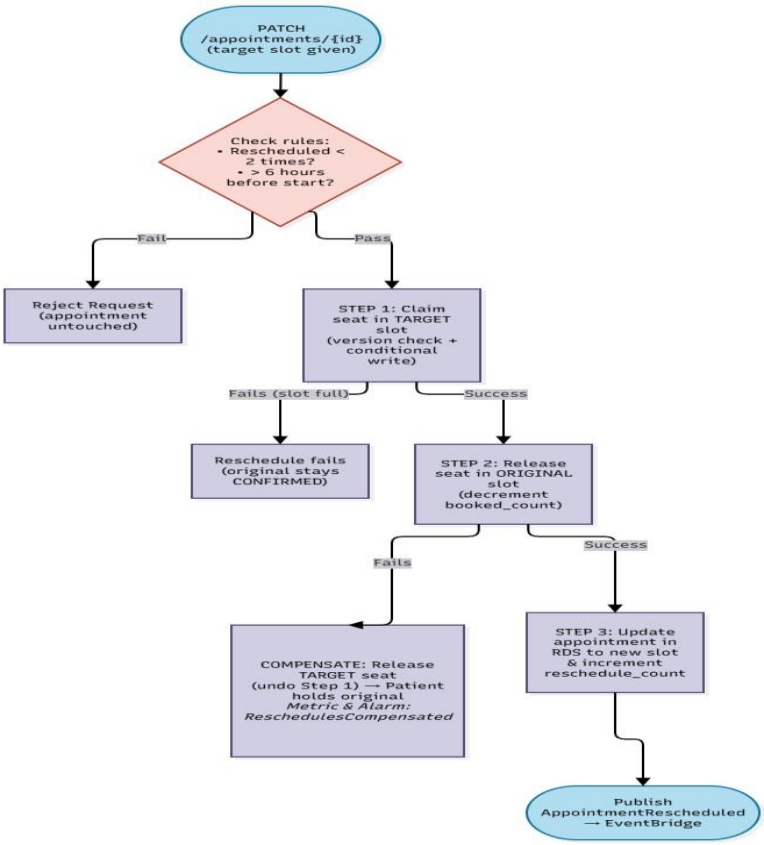


Figure: MediSlot – Appointment Rescheduling Saga and Compensation Workflow

- **Validation:** Checks rescheduling limits and requires requests to be made at least 6 hours before the appointment.
- **Claim Target Slot:** Uses a version check and conditional write to reserve a seat in the new slot.

Confidential - Not for Public Consumption or Distribution

- **Release Original Slot:** Releases the original seat after successfully claiming the target slot.
- **Update Appointment:** Updates the appointment in RDS MySQL and increments the reschedule count.
- **Compensation:** If releasing the original slot fails, the system releases the target seat and preserves the original appointment.
- **Event Publishing:** On success, publishes AppointmentRescheduled to EventBridge.

Authentication Design

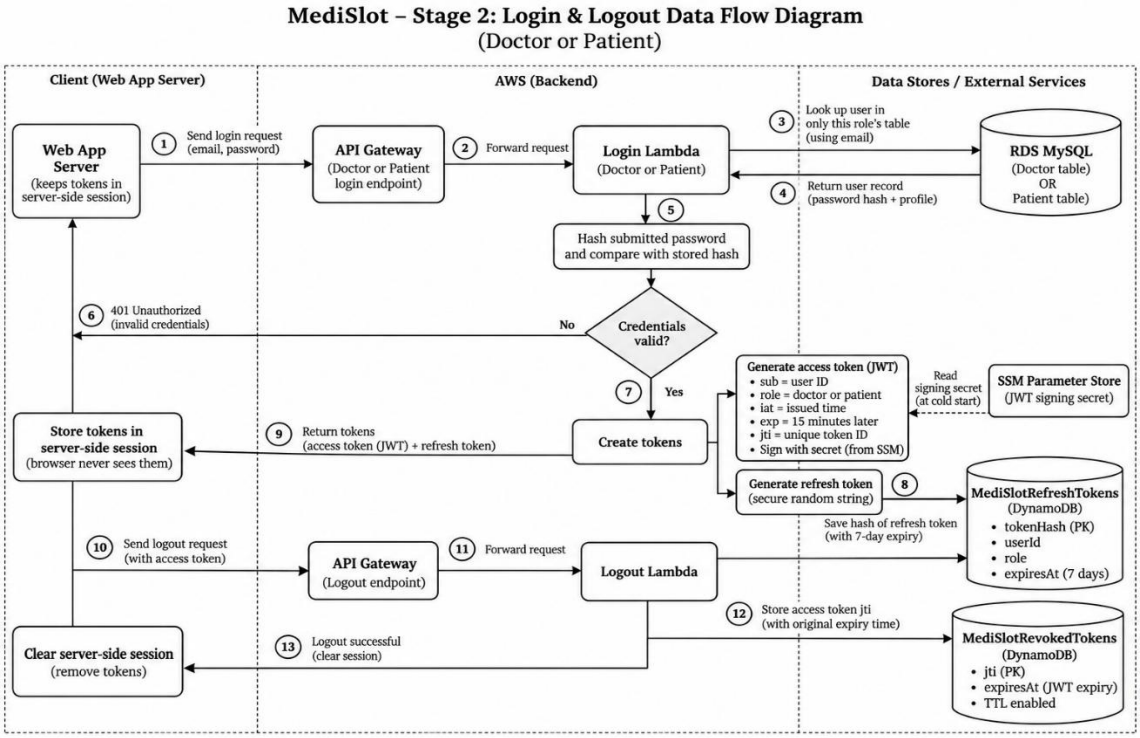


Figure: MediSlot – Login, Token Management, and Logout Data Flow Diagram

- **Login Request:** The web app sends credentials through API Gateway to the Login Lambda.
- **Credential Validation:** Login Lambda retrieves the doctor or patient record from RDS MySQL and verifies the submitted password against the stored hash.
- **Token Generation:** On successful authentication, it generates a short-lived JWT access token and a secure refresh token, using the signing secret stored in SSM Parameter Store.
- **Refresh Token Storage:** The refresh token hash, user ID, role, and expiry are stored in DynamoDB.
- **Session Management:** Tokens are stored in the server-side session and are not exposed directly to the browser.

Confidential - Not for Public Consumption or Distribution

- **Logout Process:** The Logout Lambda records the JWT ID (jti) in the revoked-token table and clears the server-side session.
- **Security:** Token expiry and revocation prevent expired or logged-out access tokens from being accepted

Registration Process

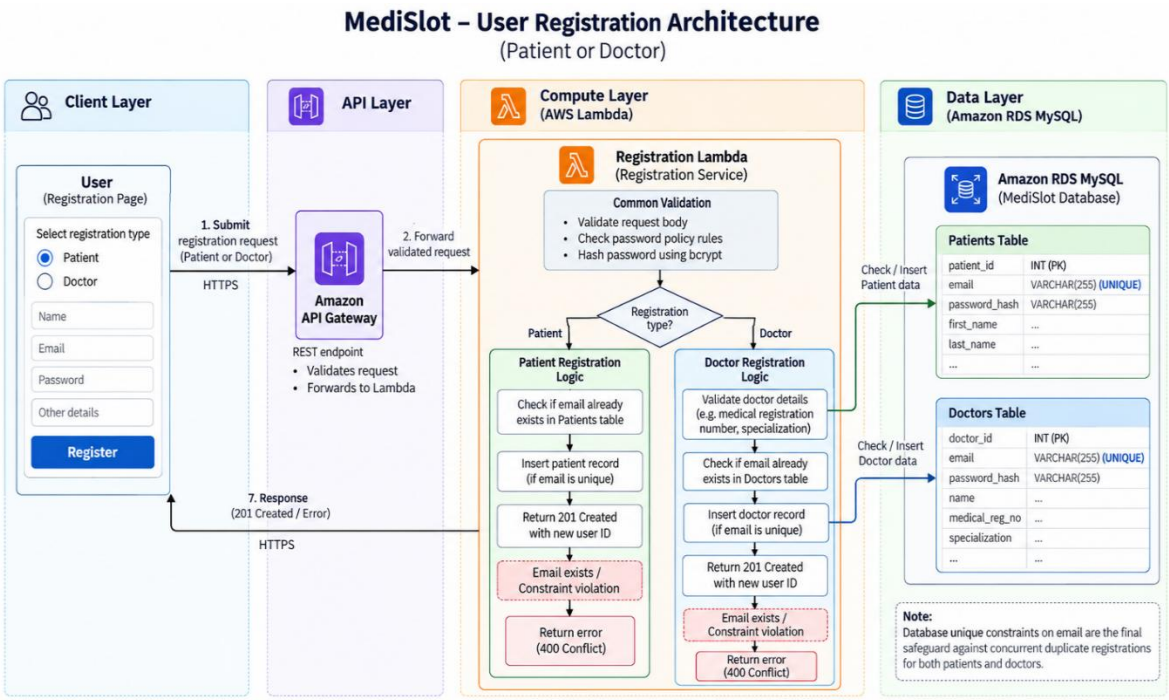


Figure: MediSlot – User Registration Architecture (Patient and Doctor)

- **Registration Request:** The user selects a patient or doctor account and submits the registration form through API Gateway over HTTPS.
- **Validation:** AWS Lambda validates the request, enforces password rules, and hashes the password using bcrypt.
- **Patient Registration:** Checks email uniqueness in the Patients table and inserts the patient record.
- **Doctor Registration:** Validates medical details, checks email uniqueness in the Doctors table, and inserts the doctor record.
- **Database:** Amazon RDS MySQL stores patient and doctor information, with unique email constraints to prevent duplicate registrations.
- **Response:** Returns 201 Created with the new user ID on success, or 400 Conflict if the email already exists.

Confidential - Not for Public Consumption or Distribution

GitHub Secrets Strategy

Item	Stored in	Notes
AWS_ROLE_ARN_DEPLOY	GitHub Environment dev	Not sensitive on its own, and usable only from master (trust policy)
AWS_REGION, ARTIFACT_BUCKET, ENV_NAME	Repository variables	Plain configuration
JWT signing secret	SSM SecureString /medislot/dev/auth/jwt_secret	Seeded once by CLI. Never in GitHub, source, or an environment variable literal
Database password	SSM SecureString	Used to access database
Database User	SSM SecureString	Database Credentials
Code File Bucket	Repository variables	Used to Save Code file

GitHub Actions CI/CD Pipeline

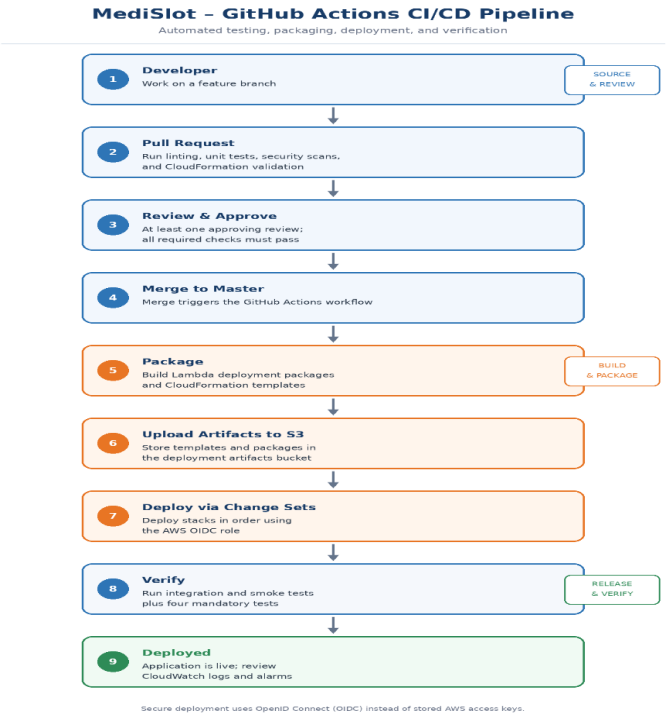


Figure: MediSlot – GitHub Actions CI/CD Pipeline

- **Development:** Developer creates changes on a feature branch.
- **Pull Request:** Linting, unit tests, security scans, and CloudFormation validation run.
- **Review and Approval:** Required reviews and checks must pass.
- **Merge:** Merging into master triggers GitHub Actions.
- **Packaging:** Lambda packages and CloudFormation templates are built.
- **Artifact Upload:** Deployment artifacts are uploaded to S3.
- **Deployment:** CloudFormation stacks are deployed using Change Sets and AWS OIDC authentication.
- **Verification:** Integration tests, smoke tests, and mandatory checks validate the deployment.
- **Completion:** The application goes live, with CloudWatch logs and alarms used for monitoring.

SSM Parameter Store Contents

Authentication

Full path	Type	Value
/medislot/{env}/auth/jwt/signing-secret	SS-manual	random 256-bit, base64
/medislot/{env}/auth/jwt/issuer	String	medislot
/medislot/{env}/auth/jwt/algorithm	String	HS256
/medislot/{env}/auth/jwt/access-ttl-seconds	String	900
/medislot/{env}/auth/jwt/refresh-ttl-seconds	String	604800
/medislot/{env}/auth/bcrypt/work-factor	String	12
/medislot/{env}/auth/password/min-length	String	10
/medislot/{env}/auth/authorizer/cache-ttl-seconds	String	60
/medislot/{env}/auth/denylist-ttl-buffer-seconds	String	0

Database

Full path	Type	Value
/medislot/{env}/db/host	Derived	RDS endpoint address
/medislot/{env}/db/port	String	3306
/medislot/{env}/db/name	String	medislot
/medislot/{env}/db/username	String	medislot_admin
/medislot/{env}/db/password	SS-manual	random

Booking

Full path	Value
/medislot/{env}/booking/window-days	30
/medislot/{env}/booking/max-upcoming-per-doctor	3
/medislot/{env}/booking/max-seats-per-patient-per-slot	1
/medislot/{env}/booking/idempotency-ttl-hours	24
/medislot/{env}/booking/retry-max-attempts	5
/medislot/{env}/booking/retry-base-backoff-ms	20
/medislot/{env}/booking/retry-max-backoff-ms	400
/medislot/{env}/booking/retry-jitter-enabled	true

Consultation types

Full path	Value
/medislot/{env}/consultation/quick/duration-min	15
/medislot/{env}/consultation/quick/capacity	1
/medislot/{env}/consultation/standard/duration-min	30
/medislot/{env}/consultation/standard/capacity	1
/medislot/{env}/consultation/detailed/duration-min	60
/medislot/{env}/consultation/detailed/capacity	1
/medislot/{env}/consultation/camp/duration-min	60
/medislot/{env}/consultation/camp/min-capacity	2
/medislot/{env}/consultation/camp/max-capacity	10

Refunds and cancellation

Full path	Value
/medislot/{env}/refund/tier1-min-hours	48
/medislot/{env}/refund/tier1-pct	100

Confidential - Not for Public Consumption or Distribution

/medislot/{env}/refund/tier2-min-hours	24
/medislot/{env}/refund/tier2-pct	75
/medislot/{env}/refund/tier3-min-hours	6
/medislot/{env}/refund/tier3-pct	50
/medislot/{env}/refund/tier4-pct	0
/medislot/{env}/refund/doctor-cancel-pct	100
/medislot/{env}/refund/noshow-pct	0
/medislot/{env}/refund/currency	INR

Reschedule

Full path	Value
/medislot/{env}/reschedule/max-count	2
/medislot/{env}/reschedule/cutoff-hours	6
/medislot/{env}/reschedule/saga-retry-attempts	3

Waitlist

Full path	Value
/medislot/{env}/waitlist/hold-duration-minutes	15
/medislot/{env}/waitlist/max-per-patient	5

Notifications and scheduled jobs

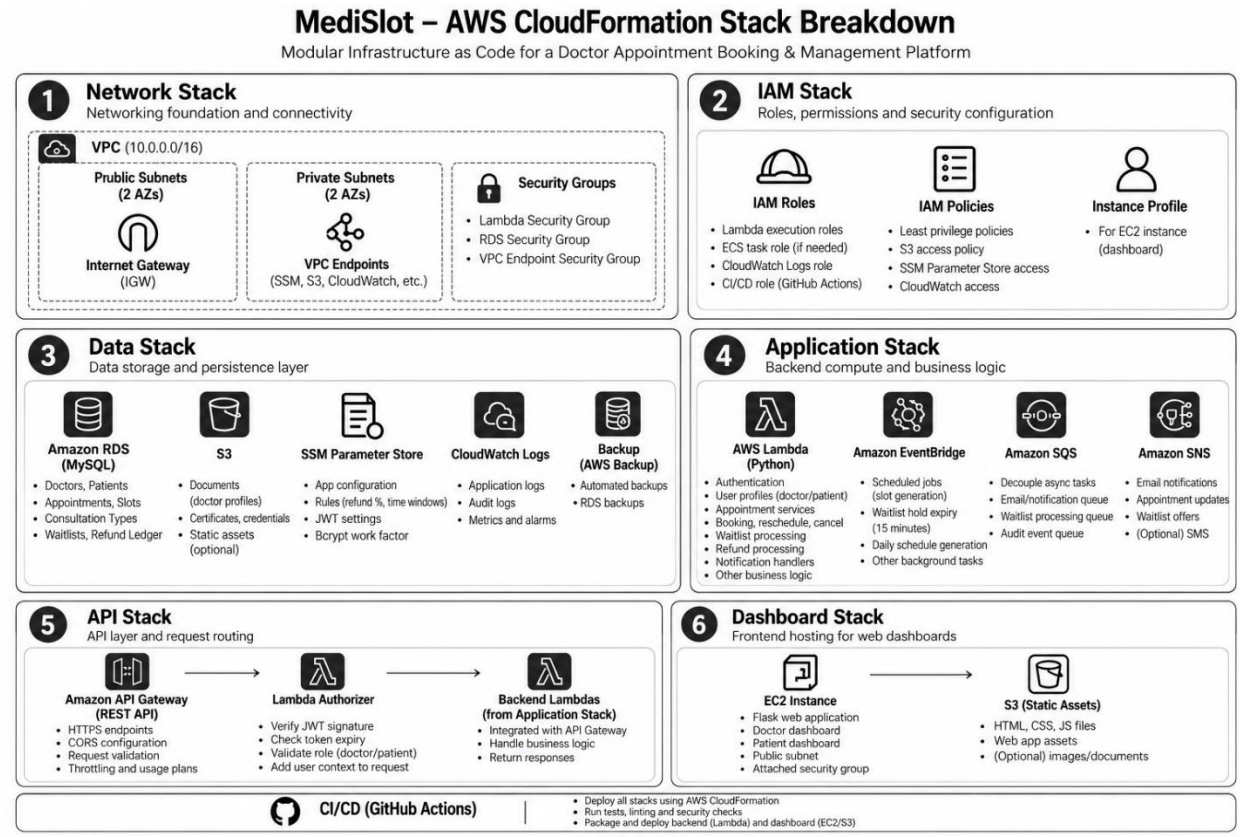
Full path	Value
/medislot/{env}/notify/reminder-1-hours	24
/medislot/{env}/notify/reminder-2-hours	2
/medislot/{env}/notify/reminder-window-minutes	5
/medislot/{env}/notify/dispatch-log-ttl-days	30
/medislot/{env}/notify/patient-topic-arn	Derived
/medislot/{env}/notify/doctor-topic-arn	Derived
/medislot/{env}/notify/ops-topic-arn	Derived
/medislot/{env}/notify/event-bus-name	Derived
/medislot/{env}/jobs/noshow-grace-minutes	30
/medislot/{env}/jobs/materialiser-rate	rate(1 day)
/medislot/{env}/jobs/offer-sweeper-rate	rate(1 mintues)
/medislot/{env}/jobs/reminder-rate	rate(5 minutes)
/medislot/{env}/jobs/noshow-rate	rate(15 minutes)

Confidential - Not for Public Consumption or Distribution

DynamoDB table names

Full path	Type
/medislot/{env}/ddb/slots-table	Derived
/medislot/{env}/ddb/waitlist-table	Derived
/medislot/{env}/ddb/hold-offers-table	Derived
/medislot/{env}/ddb/idempotency-table	Derived
/medislot/{env}/ddb/refresh-tokens-table	Derived
/medislot/{env}/ddb/token-denylist-table	Derived
/medislot/{env}/ddb/dispatch-log-table	Derived

Stacks -MediSlots



Confidential - Not for Public Consumption or Distribution

Network Stack (Networking & Security Foundation)

- **VPC (10.0.0.0/16):** Defines the isolated virtual network boundary for all platform resources.
- **Public & Private Subnets (2 AZs):** Deploys across two Availability Zones for high availability. Public subnets host internet-facing components, while private subnets isolate secure services.
- **Internet Gateway (IGW):** Enables communication between public subnets and the internet.
- **VPC Endpoints:** Provides private, secure routing to AWS services (SSM, S3, CloudWatch) without routing over the public internet.
- **Security Groups:** Functions as virtual firewalls to control inbound and outbound traffic for Lambda functions, RDS databases, and VPC Endpoints.

IAM Stack (Access & Permissions)

- **IAM Roles:** Grants secure execution permissions to AWS services (Lambda execution, ECS tasks, CloudWatch logging, and GitHub Actions CI/CD).
- **IAM Policies:** Enforces least-privilege security controls governing access to S3, SSM Parameter Store, and CloudWatch.
- **Instance Profile:** Attaches IAM roles specifically to the EC2 instances hosting the web dashboard.

Data Stack (Storage & Configuration)

- **Amazon RDS (MySQL):** Primary relational database for doctors, patients, appointments, slots, consultation types, waitlists, and refund ledgers.
- **Amazon S3:** Stores documents (doctor profiles, credentials), certificates, and optional static media.
- **SSM Parameter Store:** Manages runtime configuration keys like refund rules, JWT settings, and bcrypt factors.
- **CloudWatch Logs:** Centralized store for application logs, security audit logs, and operational metrics/alarms.
- **AWS Backup:** Automates scheduled backups for RDS databases and persistent storage

Application Stack (Backend Logic & Event Processing)

- **AWS Lambda (Python):** Handles core serverless backend operations including user profiles, appointment bookings, cancellations, waitlists, refunds, and notifications.
- **Amazon EventBridge:** Schedules recurring background jobs, such as automated slot generation, waitlist expiration timers (15 min), and daily schedule updates.
- **Amazon SQS:** Queues asynchronous messages to decouple tasks like email delivery, waitlist processing, and audit log processing.

Confidential - Not for Public Consumption or Distribution

- **Amazon SNS:** Publishes push notifications, email alerts, appointment updates, waitlist offers, and SMS messages.

API Stack (Request Routing & Authentication)

- **Amazon API Gateway (REST API):** Manages external HTTPS endpoints, CORS rules, request validation, throttling, and usage rate limits.
- **Lambda Authorizer:** Intercepts incoming requests to verify JWT signatures, check token expiration, and validate user roles (Doctor vs. Patient).
- **Backend Lambdas:** Executes the business logic passed from API Gateway and returns API responses to clients.

Dashboard Stack (Frontend Hosting)

- **EC2 Instance:** Hosts the Flask-based web application containing both doctor and patient web dashboards in a public subnet.
- **S3 (Static Assets):** Serves static frontend assets such as HTML, CSS, JavaScript, and UI images.

API Endpoints

DOCTOR

1. Authentication — shared by both roles


```
[
  {
    "method": "POST",
    "path": "/auth/refresh",
    "role": "PUBLIC",
    "requestBody": {
      "refreshToken": "string"
    },
    "response": {
      "accessToken": "<NEW_JWT>",
      "refreshToken": "<NEW_REFRESH_TOKEN>",
      "expiresIn": 900
    }
  },
  {
```

```
  "method": "POST",
  "path": "/auth/logout",
  "role": "AUTHENTICATED",
  "requestBody": {
    "refreshToken": "string"
  },
  "response": {
    "status": 204
  }
 }]
```


2. Doctor profile management


```
[
 {
  "method": "GET",
  "path": "/doctors/me",
  "role": "DOCTOR",
  "requestBody": null,
  "response": {
    "doctorId": 101,
    "email": "doctor@example.com",
    "fullName": "Dr. Rahul Sharma"
  }
 },
 {
  "method": "PUT",
  "path": "/doctors/me",
  "role": "DOCTOR",
  "requestBody": {
    "fullName": "Dr. Rahul Sharma",
    "specialisation": "Cardiology",
```


Confidential - Not for Public Consumption or Distribution

```
    "hospital": "City Hospital",
    "city": "Hyderabad"
  },
  "response": {
    "message": "Profile updated successfully"
  }
 }
]
```


3. Doctor photo — pre-signed upload URL


```
   {
    "method": "POST",
    "path": "/doctors/me/photo",
    "role": "DOCTOR",
    "requestBody": {
     "fileName": "doctor-profile.jpg",
     "contentType": "image/jpeg"
    },
    "response": {
     "uploadUrl": "<PRE_SIGNED_S3_URL>",
     "expiresIn": 300
    }
   }
```


4. Consultation types


```
   [
    {
     "method": "POST",
     "path": "/doctors/me/consultation-types",
     "role": "DOCTOR",
     "requestBody": {
      "name": "In-person consultation",
```


Confidential - Not for Public Consumption or Distribution

```
    "durationMinutes": 30,
    "price": 800
  },
  "response": {
    "typeId": 1,
    "message": "Consultation type created"
  }
},
{
  "method": "GET",
  "path": "/doctors/me/consultation-types",
  "role": "DOCTOR",
  "requestBody": null,
  "response": [
    {
      "typeId": 1,
      "name": "In-person consultation",
      "durationMinutes": 30,
      "price": 800
    }
  ]
},
{
  "method": "PUT",
  "path": "/doctors/me/consultation-types/{typeId}",
  "role": "DOCTOR",
  "requestBody": {
    "name": "Online consultation",
    "durationMinutes": 20,
    "price": 600
```


Confidential - Not for Public Consumption or Distribution

```
    },
    "response": {
      "message": "Consultation type updated"
    }
  },
  {
    "method": "DELETE",
    "path": "/doctors/me/consultation-types/{typeId}",
    "role": "DOCTOR",
    "requestBody": null,
    "response": {
      "status": 204
    }
  }
]
```

5. Weekly availability

```
{
  "method": "PUT",
  "path": "/doctors/me/availability",
  "role": "DOCTOR",
  "requestBody": {
    "timezone": "UTC",
    "weeklyPatterns": [
      {
        "dayOfWeek": "MONDAY",
        "startTime": "09:00",
        "endTime": "13:00",
        "consultationTypeIds": [1]
      },
      {
```

```
        "dayOfWeek": "WEDNESDAY",
        "startTime": "14:00",
        "endTime": "18:00",
        "consultationTypeIds": [1, 2]
      }
    ]
  },
  "response": {
    "message": "Availability updated successfully"
  }
}
```


6. Blackout dates


```
[
  {
    "method": "POST",
    "path": "/doctors/me/blackouts",
    "role": "DOCTOR",
    "requestBody": {
      "startDateTime": "2026-11-10T09:00:00+05:30",
      "endDateTime": "2026-11-10T18:00:00+05:30",
      "reason": "Personal leave"
    },
    "response": {
      "blackoutId": 12,
      "message": "Blackout created"
    }
  },
  {
    "method": "GET",
    "path": "/doctors/me/blackouts",
```


Confidential - Not for Public Consumption or Distribution

```
    "role": "DOCTOR",
    "requestBody": null,
    "response": [
      {
        "blackoutId": 12,
        "startDateTime": "2026-11-10T09:00:00+05:30",
        "endDateTime": "2026-11-10T18:00:00+05:30",
        "reason": "Personal leave"
      }
    ]
  },
  {
    "method": "DELETE",
    "path": "/doctors/me/blackouts/{blackoutId}",
    "role": "DOCTOR",
    "requestBody": null,
    "response": {
      "status": 204
    }
  }
]
```


7. Daily schedule — includes booked patients


```
{
  "method": "GET",
  "path": "/doctors/me/schedule?date=2026-11-10",
  "role": "DOCTOR",
  "requestBody": null,
  "response": {
    "date": "2026-11-10",
    "appointments": [
```


Confidential - Not for Public Consumption or Distribution

```
      {
       "appointmentId": 501,
       "startTime": "10:00",
       "endTime": "10:30",
       "status": "BOOKED",
       "patient": {
        "patientId": 201,
        "fullName": "Amit Kumar"
       }
      }
     ]
    }
   }
```


8. Doctor appointments — filter by date and status


```
   {
    "method": "GET",
    "path": "/doctors/me/appointments?from=2026-11-01&to=2026-11-30&status=BOOKED",
    "role": "DOCTOR",
    "requestBody": null,
    "response": [
     {
      "appointmentId": 501,
      "patientId": 201,
      "patientName": "Amit Kumar",
      "startTime": "2026-11-10T10:00:00+05:30",
      "status": "BOOKED"
     }
    ]
   }
```


9. Cancel a slot — refund affected patients

Confidential - Not for Public Consumption or Distribution

```
{
 "method": "DELETE",
 "path": "/doctors/me/slots/{slotId}",
 "role": "DOCTOR",
 "requestBody": null,
 "response": {
  "message": "Slot cancelled",
  "slotId": "slot_123",
  "affectedAppointments": 2,
  "refundsInitiated": 2
 }
}
```


10. Record appointment outcome


```
{
 "method": "POST",
 "path": "/doctors/me/appointments/{id}/outcome",
 "role": "DOCTOR",
 "requestBody": {
  "outcome": "COMPLETED"
 },
 "response": {
  "appointmentId": 501,
  "status": "COMPLETED",
  "message": "Appointment outcome recorded"
 }
}
```


Patients

1. Patient registration and login


```
[
 {
```


Confidential - Not for Public Consumption or Distribution

```
  "method": "POST",
  "path": "/auth/patients/register",
  "role": "PUBLIC",
  "requestBody": {
   "email": "patient@example.com",
   "password": "SecurePass123",
   "fullName": "Amit Kumar"
  },
  "response": {
   "patientId": 201,
   "message": "Patient registered successfully"
  }
 },
 {
  "method": "POST",
  "path": "/auth/patients/login",
  "role": "PUBLIC",
  "requestBody": {
   "email": "patient@example.com",
   "password": "SecurePass123"
  },
  "response": {
   "accessToken": "<JWT_ACCESS_TOKEN>",
   "refreshToken": "<OPAQUE_REFRESH_TOKEN>",
   "expiresIn": 900
  }
 }
]
```


2. Patient profile management


```
[
```


Confidential - Not for Public Consumption or Distribution

```
{
  "method": "GET",
  "path": "/patients/me",
  "role": "PATIENT",
  "requestBody": null,
  "response": {
    "patientId": 201,
    "email": "patient@example.com",
    "fullName": "Amit Kumar"
  }
},
{
  "method": "PUT",
  "path": "/patients/me",
  "role": "PATIENT",
  "requestBody": {
    "fullName": "Amit Kumar",
    "phone": "+919876543210"
  },
  "response": {
    "message": "Profile updated successfully"
  }
},
{
  "method": "DELETE",
  "path": "/patients/me",
  "role": "PATIENT",
  "requestBody": null,
  "response": {
    "status": 204
```


Confidential - Not for Public Consumption or Distribution

```
    }
   }
  ]
```


3. Search doctors with pagination


```
{
  "method": "GET",
  "path":
"/doctors?name=Rahul&hospital=City%20Hospital&specialisation=Cardiology&city=Hyder
abad&page=1&limit=10",
  "role": "PUBLIC",
  "requestBody": null,
  "response": {
   "items": [
    {
     "doctorId": 101,
     "fullName": "Dr. Rahul Sharma",
     "specialisation": "Cardiology",
     "hospital": "City Hospital",
     "city": "Hyderabad"
    }
   ],
   "pagination": {
    "page": 1,
    "limit": 10,
    "totalItems": 1,
    "totalPages": 1
   }
  }
}
```


4. Get public doctor profile and consultation fees

Confidential - Not for Public Consumption or Distribution

```
{
 "method": "GET",
 "path": "/doctors/{doctorId}",
 "role": "PUBLIC",
 "requestBody": null,
 "response": {
  "doctorId": 101,
  "fullName": "Dr. Rahul Sharma",
  "specialisation": "Cardiology",
  "hospital": "City Hospital",
  "city": "Hyderabad",
  "consultationTypes": [
   {
    "typeId": 1,
    "name": "In-person consultation",
    "durationMinutes": 30,
    "fee": 800
   },
   {
    "typeId": 2,
    "name": "Online consultation",
    "durationMinutes": 20,
    "fee": 600
   }
  ]
 }
}
```


5. Get available appointment slots


```
{
 "method": "GET",
```


Confidential - Not for Public Consumption or Distribution

```
  "path": "/doctors/{doctorId}/slots?from=2026-11-01&to=2026-11-07&consultationTypeId=1",
  "role": "PUBLIC",
  "requestBody": null,
  "response": {
    "doctorId": 101,
    "slots": [
      {
        "slotId": "slot_abc123",
        "startTime": "2026-11-02T09:00:00Z",
        "endTime": "2026-11-02T09:30:00Z",
        "consultationTypeId": 1,
        "fee": 800
      }
    ]
  }
}
```


6. Book an appointment

The Idempotency-Key header is required to prevent duplicate bookings if the client retries the request.


```
{
  "method": "POST",
  "path": "/appointments",
  "role": "PATIENT",
  "headers": {
    "Authorization": "Bearer <JWT_ACCESS_TOKEN>",
    "Idempotency-Key": "booking-unique-key-123"
  },
  "requestBody": {
    "slotId": "slot_abc123",
```


Confidential - Not for Public Consumption or Distribution

```
      "consultationTypeId": 1
    },
    "response": {
      "appointmentId": 501,
      "status": "BOOKED",
      "slotId": "slot_abc123",
      "message": "Appointment booked successfully"
    }
  }
```


7. Get the patient's upcoming and past appointments


```
{
 "method": "GET",
 "path": "/appointments",
 "role": "PATIENT",
 "requestBody": null,
 "response": {
  "upcoming": [
   {
    "appointmentId": 501,
    "doctorId": 101,
    "doctorName": "Dr. Rahul Sharma",
    "startTime": "2026-11-02T09:00:00Z",
    "status": "BOOKED"
   }
  ],
  "past": [
   {
    "appointmentId": 450,
    "doctorId": 102,
    "doctorName": "Dr. Priya Verma",
```


Confidential - Not for Public Consumption or Distribution

```
      "startTime": "2026-10-10T10:00:00Z",
      "status": "COMPLETED"
    }
   ]
  }
}
```


8. Get appointment details


```
{
  "method": "GET",
  "path": "/appointments/{id}",
  "role": "PATIENT",
  "requestBody": null,
  "response": {
    "appointmentId": 501,
    "patientId": 201,
    "doctorId": 101,
    "doctorName": "Dr. Rahul Sharma",
    "startTime": "2026-11-02T09:00:00Z",
    "endTime": "2026-11-02T09:30:00Z",
    "status": "BOOKED",
    "fee": 800
  }
}
```


9. Reschedule an appointment


```
{
  "method": "PATCH",
  "path": "/appointments/{id}",
  "role": "PATIENT",
  "requestBody": {
    "targetSlotId": "slot_xyz789"
```


Confidential - Not for Public Consumption or Distribution

```
  },
  "response": {
    "appointmentId": 501,
    "status": "BOOKED",
    "slotId": "slot_xyz789",
    "message": "Appointment rescheduled successfully"
  }
}
```


10. Cancel an appointment and calculate refund


```
{
  "method": "DELETE",
  "path": "/appointments/{id}",
  "role": "PATIENT",
  "requestBody": null,
  "response": {
    "appointmentId": 501,
    "status": "CANCELLED",
    "refundAmount": 800,
    "currency": "INR",
    "refundStatus": "INITIATED"
  }
}
```


11. Join a slot waitlist


```
{
  "method": "POST",
  "path": "/slots/{slotId}/waitlist",
  "role": "PATIENT",
  "requestBody": {},
  "response": {
    "waitlistId": "waitlist_123",
```


Confidential - Not for Public Consumption or Distribution

```
    "slotId": "slot_abc123",
    "status": "WAITING",
    "message": "Added to waitlist successfully"
  }
}
```


12. Leave a slot waitlist


```
{
  "method": "DELETE",
  "path": "/slots/{slotId}/waitlist",
  "role": "PATIENT",
  "requestBody": null,
  "response": {
    "status": 204
  }
}
```


13. Get the patient's waitlists


```
{
  "method": "GET",
  "path": "/patients/me/waitlists",
  "role": "PATIENT",
  "requestBody": null,
  "response": [
    {
      "waitlistId": "waitlist_123",
      "slotId": "slot_abc123",
      "doctorId": 101,
      "status": "WAITING",
      "createdAt": "2026-10-23T10:00:00Z"
    }
  ]
```


Confidential - Not for Public Consumption or Distribution

```
}
```


14. Accept a waitlist offer


```
{
 "method": "POST",
 "path": "/waitlist-offers/{offerId}/accept",
 "role": "PATIENT",
 "requestBody": {},
 "response": {
  "appointmentId": 502,
  "status": "BOOKED",
  "message": "Waitlist offer accepted successfully"
 }
}
```


15. Get the patient's refund ledger


```
{
 "method": "GET",
 "path": "/patients/me/refunds",
 "role": "PATIENT",
 "requestBody": null,
 "response": {
  "refunds": [
   {
    "refundId": 301,
    "appointmentId": 501,
    "amount": 800,
    "currency": "INR",
    "status": "COMPLETED",
    "createdAt": "2026-11-02T10:00:00Z"
   }
  ]
```


Confidential - Not for Public Consumption or Distribution

}

}

Data Flow

Doctor-Pov

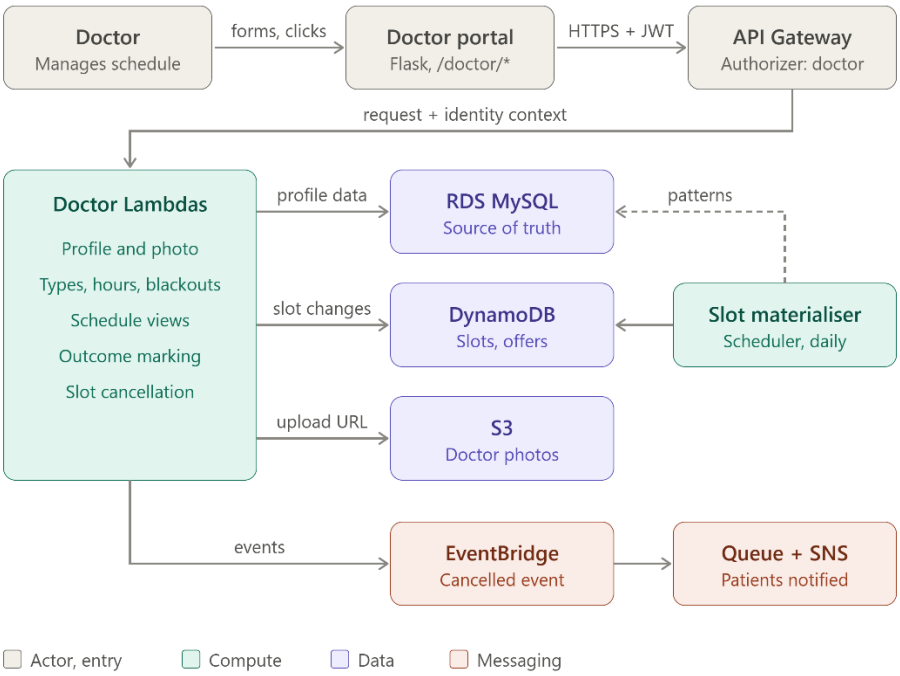


Figure : MediSlot Doctor Data Flow & Operations Architecture Diagram

1. User Interaction & Entry

- **Doctor:** Interacts with the interface via form submissions and clicks to manage schedules, availability, and profile settings.
- **Doctor Portal (Flask,** /doctor/*): Serves as the web dashboard for doctors, capturing user actions and dispatching secure HTTPS requests containing JWT credentials.
- **API Gateway (Authorizer:** doctor**):** Authenticates the incoming HTTPS request and attaches the verified identity context before forwarding it to downstream microservices.

2. Core Processing (Doctor Lambdas)

The request and identity context route to **Doctor Lambdas**, which execute backend compute operations for:

- **Profile and photo:** Profile updates and pre-signed S3 URL generation.
- **Types, hours, blackouts:** Consultation fee structures, weekly availability patterns, and holiday blackouts.

Confidential - Not for Public Consumption or Distribution

- **Schedule views:** Fetching daily slot and patient appointment schedules.
- **Outcome marking:** Marking appointments as completed or no-show.
- **Slot cancellation:** Processing doctor-initiated slot cancellations.

3. Data Persistence Layer

- **RDS MySQL (Source of Truth):** Stores core relational profile data, availability patterns, consultation types, and appointment outcomes.
- **DynamoDB (Slots, Offers):** Handles real-time slot state changes and hold offers.
- **S3 (Doctor Photos):** Directly receives uploaded profile media using pre-signed upload URLs generated by the Lambdas.

4. Background Processing & Messaging

- **Slot Materialiser (Scheduler, daily):** Reads availability patterns from RDS MySQL to automatically pre-generate and update future slots in DynamoDB on a scheduled daily basis.
- **EventBridge (Cancelled event):** Receives domain events (e.g., slot or appointment cancellations) emitted by Doctor Lambdas.
- **Queue + SNS (Patients notified):** Consumes cancellation events from EventBridge to queue and publish real-time notifications directly to affected patients.

Patient-Pov

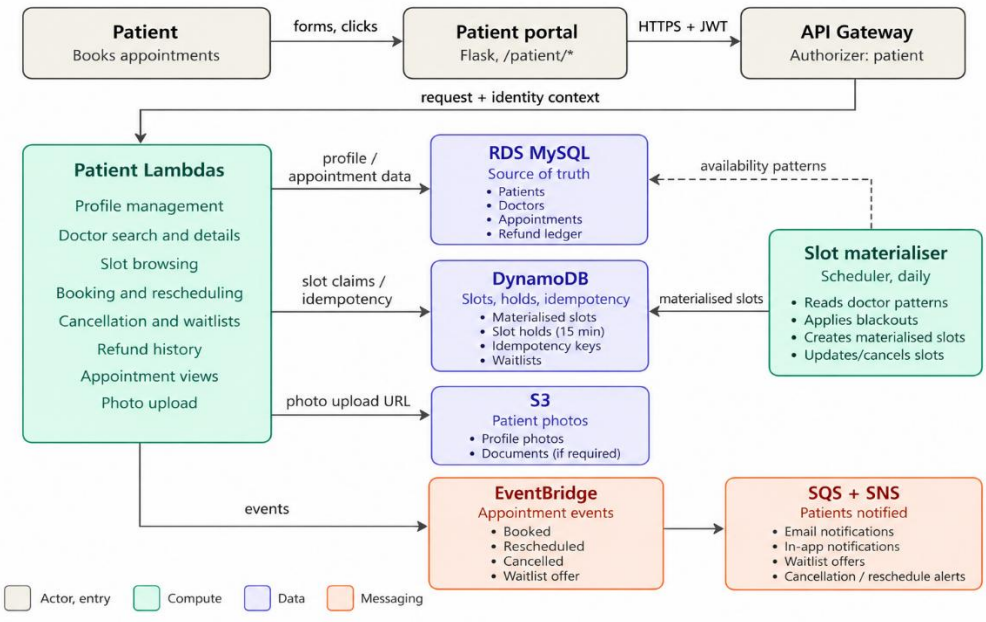


Figure Name: MediSlot Patient Data-Flow Architecture Diagram

Confidential - Not for Public Consumption or Distribution

1. User Interaction & Entry

- **Patient:** Interacts with the platform via forms and clicks to search for doctors, browse slots, and manage appointment bookings.
- **Patient Portal (Flask, /patient/*):** Acts as the web frontend interface, capturing user requests and sending HTTPS requests with JWT tokens.
- **API Gateway (Authorizer: patient):** Authenticates the incoming HTTPS request and passes the validated request with identity context to the backend.

2. Core Processing (Patient Lambdas)

The request and identity context are routed to Patient Lambdas, which execute microservice functions for:

- **Profile management:** Updating personal details and emergency contacts.
- **Doctor search and details:** Filtering and retrieving doctor profiles and consultation types.
- **Slot browsing:** Fetching real-time slot availability.
- **Booking and rescheduling:** Executing atomic slot reservations and triggering reschedule sagas.
- **Cancellation and waitlists**: Handling patient cancellations and managing waitlist queue registrations.
- **Refund history:** Fetching status and ledger details for refunded transactions.
- **Appointment views**: Displaying current, past, and upcoming appointments.
- **Photo upload:** Requesting pre-signed S3 upload URLs for profile picture updates.

3. Data Persistence Layer

- **RDS MySQL (Source of Truth):** Relational store housing persistent data for patients, doctors, appointments, and refund ledgers.
- **DynamoDB (Slots, holds, idempotency):** High-speed NoSQL database storing materialized slots, temporary slot holds (15 min), idempotency keys, and waitlist queues.
- **S3 (Patient Photos):** Receives direct image and optional document uploads using pre-signed upload URLs.

4. Background Processing & Messaging

- **Slot Materialiser (Scheduler, daily):** Background service that reads availability patterns from RDS MySQL, applies blackout dates, creates materialized slots, and updates/cancels slots in DynamoDB.
- **EventBridge (Appointment events**): Receives system lifecycle events emitted by Patient Lambdas, including:
    - Booked: New appointment created.

Confidential - Not for Public Consumption or Distribution

- Rescheduled: Slot shifted to a new time.
- Cancelled: Booking cancelled by patient.
- Waitlist offer: Automatic hold offer generated for next waitlisted patient.

- **SQS + SNS (Patients notified):** Consumes lifecycle events to queue and publish patient alerts via email notifications, in-app notifications, waitlist offers, and cancellation/reschedule alerts.